Enhancing Accessibility of a Web-based Scheme IDE to Improve Inclusion in Computing

CMDiT Fostering Innovation Through Inclusion

Student Poster Competition

Jacob Bell, ***Nicole Moreno González,*** *Charlotte Wade*

Grinnell College

Introduction

Accessibility is not just a design preference: it's a legal requirement in many places and a prerequisite for equitable participation in education [1]. For blind and low vision (BLV) and neurodiverse (ND) learners, inaccessible software can hurt both learning and confidence in programming. In this project, we examine the accessibility of Scamper, a Scheme-based IDE for introductory computer science. While pedagogically strong, our manual audit with screen readers found that Scamper's interface posed major barriers for BLV and ND users. We address these issues and extend the interface with features designed to better support these learners; features that, following Universal Design, also help sighted and neurotypical users.

Design Pitfalls and Solutions

Limited code navigation support: Scamper didn't offer a way to navigate Scheme's nested syntax, so screen readers couldn't convey structure. This made it hard, especially for BLV users, to locate expressions or stay oriented in larger programs. To help with this, we added two optional, interactive abstract syntax tree (AST) views. There are different versions to accommodate for different user preferences. The AST nodes are screen-reader-friendly, show nesting clearly, and lets users click nodes to jump to and edit the related code in the editor.

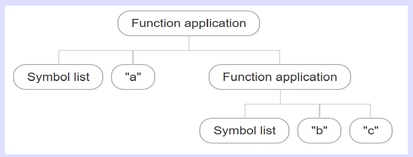
```
1  ; example.scm
2
3  (list "a" (list "b" "c"))
```

An example of a nested list shown in the form of an ASCII AST and a traditional tree node AST.

Abstract Syntax Tree

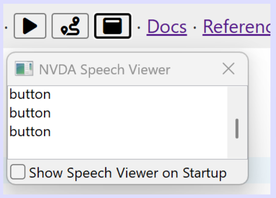
```
└─ Function application
   ├─ Symbol list
   │  ├─ "a"
   │  └─ Function application
   │     ├─ Symbol list
   │     │  ├─ "b"
   │     │  └─ "c"
```

A more textual AST

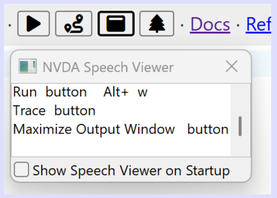


A more visual AST

Missing semantic labeling: Key buttons like "Run" and "Trace" lacked Accessible Rich Internet Applications (ARIA) attributes, making them invisible to screen readers [2]. We addressed this by adding appropriate ARIA-labels and role attributes to all interactive elements, so screen readers can now identify, describe, and navigate them reliably.

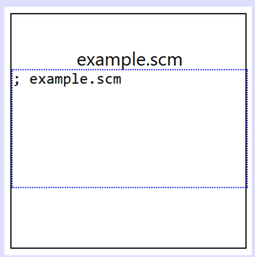


Buttons without appropriate labels for Screen Reader navigation.

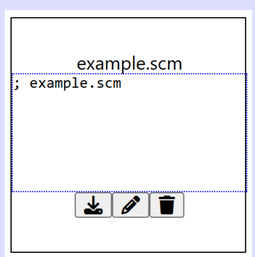


Buttons with meaningful labels (including keyboard commands) for Screen Reader navigation.

Hover-dependent functionality: Scamper included controls like download, rename, and delete that only appeared on hover. Since hover states aren't keyboard-accessible, these controls remained hidden from assistive tech. We replaced all hover-only features with always-visible controls that stay unobtrusive but are fully accessible via keyboard and screen reader.



Save, rename and delete buttons are hidden. These are inaccessible by Screen Readers.



Buttons always on display. They are accessible by Screen Readers.

Code formatting: While BLV users could write valid code, the lack of indentation feedback makes it hard to follow standard formatting. This could affect collaboration with sighted and neurodiverse peers and instructors. We introduced a keyboard-accessible auto-indentation feature that formats code consistently without disrupting the user's cursor or workflow.

```
(define drop-last
              (lambda (lst)
(if (null? (cdr lst))
          null
(cons (car lst) (drop-last (cdr lst))))))
```

Working code with inconsistent indentation.

```
(define drop-last
  (lambda (lst)
    (if (null? (cdr lst))
      null
      (cons (car lst) (drop-last (cdr lst))))))
```

Working code with correct indentation after using auto-indent command.

Conclusions

Accessibility is not a one-time fix, but an ongoing process. Through this project, we uncovered how small oversights, like missing ARIA labels or hover-only buttons, can create outsized barriers for BLV and ND students. Retrofitting accessibility, especially in features like the AST or keyboard navigation, not only removed barriers, but also improved usability for everyone, reaffirming the value of Universal Design. By navigating Scamper with assistive technologies ourselves, we gained insights that guidelines alone could not provide. This work reminds us that accessibility is best achieved through deliberate testing, structural support, ongoing attention to the needs of all learners, and taking accessibility considerations into account from the beginning. While we have addressed some disabilities, continuing this work to support other disabilities is imperative.

References

[1] Natalie L. Shaheen. 2022. Technology accessibility: How U.S. K-12 schools are enacting policy and addressing the equity imperative. *Computers Education* 179 (April 2022). https://doi.org/10.1016/j.compedu.2021.104414

[2] W3C. 2021. Accessible Rich Internet Applications (WAI-ARIA) 1.2. World Wide Web Consortium (W3C). December 16, 2021. https://www.w3.org/TR/wai-aria-1.2/